

前端开发工程师知识库

从入门到精通的职业技能培训教材

本教材涵盖 HTML5、CSS3、JavaScript、ES6+、React、前端工程化、性能优化与安全的完整知识体系，面向前端开发职业技能培训。

目录

第1章 HTML5 语义化与文档结构

1.1 HTML5 语义化标签与文档结构

1.2 HTML5 表单与多媒体元素

第2章 CSS 选择器与盒模型

2.1 CSS 选择器与优先级计算

2.2 盒模型与布局基础

第3章 现代 CSS 布局技术

3.1 Flexbox 弹性布局

3.2 Grid 网格布局

第4章 响应式设计 with 媒体查询

4.1 响应式设计原理与断点策略

4.2 媒体查询与流体布局

第5章 JavaScript 核心基础

5.1 变量、数据类型与运算符

5.2 流程控制、函数与作用域

第6章 ES6+ 新特性与异步编程

6.1 ES6+ 语法新特性

6.2 异步编程与 Promise

第7章 React 框架基础

7.1 React 组件与 JSX

7.2 Props、State 与 Hooks

第8章 前端工程化与性能优化

8.1 构建工具与包管理

8.2 前端性能优化与安全

第1章 HTML5 语义化与文档结构

1.1 HTML5 语义化标签与文档结构

【知识点讲解】

HTML5 语义化是指使用恰当的 HTML 标签来描述内容的含义，而非仅仅关注外观表现。语义化的核心在于让标签本身携带信息，使浏览器、搜索引擎和辅助技术（如屏幕阅读器）能够正确理解页面结构。

HTML5 引入了一系列语义化标签，包括：`<header>`、`<nav>`、`<main>`、`<article>`、`<section>`、`<aside>`、`<footer>`、`<figure>`、`<figcaption>`、`<time>` 等。

【操作步骤或工作流程】

- 步骤1：分析页面结构，识别头部、导航、主体内容、侧边栏、底部等区域。
- 步骤2：使用 `<header>` 包裹页面头部，内部放置 `<h1>` 站点标题和 `<nav>` 主导航。
- 步骤3：使用 `<main>` 包裹页面主体内容，确保每个页面只有一个 `<main>`。
- 步骤4：在 `<main>` 内部，使用 `<article>` 包裹独立的内容块（如每篇博客文章）。
- 步骤5：在 `<article>` 内部，使用 `<section>` 划分不同的内容节，每个 section 配一个标题。
- 步骤6：使用 `<aside>` 包裹侧边栏内容，如相关文章推荐、广告等。
- 步骤7：使用 `<footer>` 包裹页面底部，放置版权信息和辅助链接。
- 步骤8：使用 `<time datetime="2024-01-15">2024年1月15日</time>` 标注发布时间。

【注意事项】

1. `<main>` 标签在一个页面中只能使用一次，且不能嵌套在 `<article>`、`<aside>`、`<footer>`、`<header>` 或 `<nav>` 内部。
2. `<section>` 不是通用的容器，它表示文档中的一个主题区块，通常应包含标题。
3. `<article>` 强调内容的独立性，如果内容脱离当前页面仍然有意义，应使用 `<article>`。
4. `<div>` 和 `<span>` 是无语义标签，仅在纯粹需要布局容器且无合适语义标签时使用。
5. 语义化标签自带合理的默认样式，但不应依赖默认样式进行视觉设计。

【常见问题】

Q1：使用 `<div class="header">` 和 `<header>` 有什么区别？

A：视觉效果可以完全相同，但 `<header>` 对屏幕阅读器和搜索引擎更友好，能自动识别为页面头部区域，提升可访问性和 SEO。

Q2：一个页面可以有多个 `<header>` 吗？

A：可以。`<header>` 既可以作为页面级别的头部，也可以作为 `<article>` 或 `<section>` 的头部。

Q3：`<section>` 和 `<div>` 如何选择？

A：如果内容是一个独立的主题区块且有标题，用 `<section>`；如果只是需要布局容器，用 `<div>`。

【实际案例】

案例：新闻网站文章页面结构

```
<!DOCTYPE html>

<html lang="zh-CN">

<head>

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>科技新闻 - 每日报道</title>

</head>

<body>

<header>

<h1>每日报道</h1>

<nav>

<ul>

<li><a href="#tech">科技</a></li>

<li><a href="#finance">财经</a></li>

<li><a href="#sports">体育</a></li>

</ul>
```

</nav>

</header>

<main>

<article>

<header>

<h2>人工智能技术在医疗领域的突破</h2>

<p>发布于 <time datetime="2024-03-15">2024年3月15日</time></p>

</header>

<section>

<h3>技术背景</h3>

<p>近年来，深度学习技术在图像识别领域取得了显著进展...</p>

</section>

<section>

<h3>临床应用</h3>

<p>多家医院已引入 AI 辅助诊断系统，准确率提升至 95% 以上...</p>

</section>

<footer>

<p>作者：张科技 | 责任编辑：李编辑</p>

</footer>

</article>

<aside>

<h3>相关推荐</h3>

AI 芯片市场分析

智慧医院建设指南

</aside>

```
</main>

<footer>

<p>&copy; 2024 每日报道. 保留所有权利。</p>

</footer>

</body>

</html>
```

1.2 HTML5 表单与多媒体元素

【知识点讲解】

HTML5 对表单和多媒体进行了大幅增强，提供了更丰富的输入类型和原生多媒体支持。

表单增强包括：新输入类型（email、tel、url、number、range、date、time、color、search）、新属性（placeholder、required、pattern、min、max、step、autofocus、autocomplete）、新元素（<datalist>、<output>、<progress>、<meter>）。

多媒体元素：<audio>、<video>、<source>、<track>。

【操作步骤或工作流程】

步骤1：根据数据类型选择合适的 input 类型，如邮箱用 type="email"，手机号用 type="tel"。

步骤2：添加 required 属性标记必填字段，添加 placeholder 提供输入提示。

步骤3：使用 pattern 属性配合正则表达式进行格式验证。

步骤4：使用 <datalist> 为输入框提供自动补全建议列表。

步骤5：使用 <audio> 或 <video> 标签嵌入多媒体，内部放置多个 <source> 提供格式备选。

步骤6：为视频添加 <track> 字幕，使用 kind="subtitles" 和 srclang 属性。

步骤7：使用 JavaScript 控制多媒体播放，如 play()、pause()、currentTime 等。

【注意事项】

1. HTML5 表单验证是客户端验证，不能替代服务端验证，恶意用户可绕过。
2. <input type="date"> 在不同浏览器中的 UI 表现不一致，需要统一样式处理。

3. 多媒体元素应始终提供多个 `<source>`，因为不同浏览器支持的格式不同。
4. 视频应提供 `poster` 属性作为封面图，提升加载体验。
5. 自动播放（`autoplay`）在移动端通常被浏览器阻止，需要用户交互触发。

【常见问题】

Q1：为什么设置了 `required` 但表单仍能提交？

A：如果通过 JavaScript 调用 `form.submit()` 而不是点击提交按钮，HTML5 验证不会触发。应使用 `form.checkValidity()` 手动验证。

Q2：如何自定义 `<input type="file">` 的样式？

A：原生文件输入框样式难以自定义，通常做法是将 `input` 隐藏，然后用自定义按钮触发其 `click()` 事件。

Q3：视频无法播放怎么办？

A：检查视频编码格式是否为

H.264（MP4），这是兼容性最好的格式。同时确保服务器正确配置了

MIME

类型（`video/mp4`）。

【实际案例】

案例：用户注册表单与视频播放器

```
<form action="/register" method="POST" novalidate>

<label for="email">邮箱：</label>

<input type="email" id="email" name="email" placeholder="example@mail.com" required>

<label for="phone">手机号：</label>

<input      type="tel"      id="phone"      name="phone"      pattern="1[3-9][0-9]{9}"
placeholder="13800138000" required>

<label for="birth">出生日期：</label>

<input type="date" id="birth" name="birth" min="1900-01-01" max="2024-12-31">

<label for="city">城市：</label>

<input list="cities" id="city" name="city" placeholder="选择或输入城市">

<datalist id="cities">
```

<option value="北京">

<option value="上海">

<option value="广州">

<option value="深圳">

</datalist>

<button type="submit">注册</button>

</form>

<video width="640" height="360" controls poster="cover.jpg">

<source src="video.mp4" type="video/mp4">

<source src="video.webm" type="video/webm">

<track kind="subtitles" src="subtitles_zh.vtt" srclang="zh" label="中文字幕">

您的浏览器不支持视频播放。

</video>

第2章 CSS 选择器与盒模型

2.1 CSS 选择器与优先级计算

【知识点讲解】

CSS 选择器用于定位需要设置样式的 HTML 元素。理解选择器和优先级是编写可维护 CSS 的基础，也是排查样式冲突的关键技能。

基础选择器：元素选择器 (p、div)、类选择器 (.class)、ID 选择器 (#id)、通配符 (*)、属性选择器 ([type="text"])。

组合选择器：后代 (A B)、子代 (A > B)、相邻兄弟 (A + B)、通用兄弟 (A ~ B)、群组 (A, B)。

伪类与伪元素：:hover、:focus、:active、:nth-child()、:first-child、:last-child、:not()、:is()、:where()、::before、::after。

优先级计算采用 (a, b, c) 格式：a = ID 选择器数量，b = 类选择器 + 伪类 + 属性选择器数量，c = 元素选择器 + 伪元素数量。比较时从左到右逐位比较。内联样式优先级高于 ID 选择器，!important 可覆盖任何优先级但应谨慎使用。

【操作步骤或工作流程】

步骤1：分析需要选中的元素，优先使用低优先级的选择器（如类选择器），便于后续覆盖。

步骤2：需要选中直接子元素时使用 >，避免选中嵌套更深的无关元素。

步骤3：使用 :nth-child() 实现交替样式，如 :nth-child(odd) 和 :nth-child(even)。

步骤4：使用 :not() 排除特定元素，如 input:not([type="submit"])。

步骤5：使用 :is() 简化复杂选择器，如 :is(h1, h2, h3) 同时选中多个标题。

步骤6：遇到样式冲突时，计算双方优先级 (a,b,c)，优先级高的生效。

步骤7：避免使用 !important，如果必须使用，应在注释中说明原因。

【注意事项】

1. 选择器越具体，优先级越高，但过度具体会降低可维护性，建议控制在三层以内。

2. `:where()` 的优先级始终为 0，适合用于重置样式。
3. 内联样式（`style` 属性）优先级为 (1,0,0,0)，高于任何选择器。
4. `!important` 会打破样式的自然层叠，应作为最后手段。
5. 相同优先级的样式，后定义的规则会覆盖先定义的规则（层叠特性）。

【常见问题】

Q1：为什么我的样式没有生效？

A：首先检查选择器是否正确匹配目标元素，然后计算优先级是否被其他规则覆盖，最后检查是否有拼写错误。

Q2：如何快速提高选择器优先级？

A：增加类选择器或嵌套层级，如 `.btn` 改为 `.nav .btn`。更好的做法是重构 CSS 结构，避免优先级军备竞赛。

Q3：`:is()` 和 `:where()` 有什么区别？

A：`:is()` 的优先级等于其参数中优先级最高的选择器；`:where()` 的优先级始终为 0。

【实际案例】

案例：电商网站商品列表样式

```
.product-card {  
  border: 1px solid #e0e0e0;  
  border-radius: 8px;  
  padding: 16px;  
}  
  
.product-list > .product-card {  
  margin-bottom: 20px;  
}  
  
.product-title + .product-price {  
  font-weight: bold;  
  color: #e53935;
```

```
}

.review-list > .review-item:nth-child(odd) {

background: #f5f5f5;

}

.product-card:not(.out-of-stock) {

cursor: pointer;

}

.text-red { color: red; }

p.text-red { color: blue; }

#special-price { color: green; }
```

2.2 盒模型与布局基础

【知识点讲解】

CSS 盒模型是理解页面布局的核心。每个元素都被视为一个矩形盒子，由四部分组成：Content（内容）、Padding（内边距）、Border（边框）、Margin（外边距）。

box-sizing 属性：content-box（默认）width/height 只包含 content，padding 和 border 额外增加；border-box width/height 包含 content + padding + border，更直观易控。

块级元素 vs 行内元素：块级（display: block）独占一行可设宽高；行内（display: inline）不独占一行不可设宽高；行内块（display: inline-block）可设宽高且不独占一行。

外边距折叠（Margin Collapse）：垂直相邻的块级元素的外边距会合并，取较大值而非相加。

BFC（Block Formatting Context）：创建方式包括 overflow 不为 visible、float 不为 none、display 为 inline-block/table-cell、position 为 absolute/fixed。作用：阻止外边距折叠、包含浮动元素、阻止元素被浮动元素覆盖。

【操作步骤或工作流程】

步骤1：统一设置全局 box-sizing: border-box，使尺寸计算更直观。

步骤2：设计布局时先确定元素的 display 类型：块级、行内还是行内块。

步骤3：设置 width 和 height 时，border-box 模式下直接写目标尺寸即可。

步骤4：使用 margin: 0 auto 实现块级元素水平居中。

步骤5：遇到垂直间距异常时，检查是否发生了外边距折叠。

步骤6：需要阻止外边距折叠时，给父元素添加 overflow: hidden 或 border/padding。

步骤7：处理浮动布局时，使用 clearfix 或创建 BFC 清除浮动。

【注意事项】

1. 行内元素的垂直 margin 不生效，垂直 padding 虽会渲染背景但不会占据布局空间。
2. 水平方向的外边距不会发生折叠，只有垂直方向会。
3. 空元素的上下外边距也会发生折叠。
4. 浮动元素、绝对定位元素、行内块元素之间不会发生外边距折叠。
5. 百分比 width/height 是相对于包含块计算的，不是相对于视口。

【常见问题】

Q1：设置了 width: 100% 为什么还超出了父容器？

A：如果使用的是 content-box 且元素有 padding 或 border，总宽度会超出 100%。改用 border-box 即可解决。

Q2：两个 div 之间的间距不是 margin 之和？

A：发生了外边距折叠，取较大值。给其中一个元素添加 padding 或 border 可阻止折叠。

Q3：如何实现垂直居中？

A：单行文本可用 line-height 等于容器高度；块级元素可用 Flexbox 或 Grid 布局；已知高度可用绝对定位 + margin 负值。

【实际案例】

案例：卡片组件的盒模型设计

```
*, *::before, *::after {  
  
  box-sizing: border-box;  
  
}
```

```
.card {  
  
width: 300px;  
  
padding: 20px;  
  
border: 2px solid #333;  
  
margin: 0 auto;  
  
background: #fff;  
  
}  
  
.card-content {  
  
width: 100%;  
  
}  
  
.article {  
  
margin-bottom: 30px;  
  
}  
  
.article + .article {  
  
margin-top: 20px;  
  
}  
  
.article-wrapper {  
  
overflow: hidden;  
  
}
```

第3章 现代 CSS 布局技术

3.1 Flexbox 弹性布局

【知识点讲解】

Flexbox（弹性盒子）是一维布局模型，专门用于在一个方向上（行或列）排列元素。它解决了传统布局中垂直居中、等高列、底部固定等难题，是现代前端开发的首选布局方案。

核心概念：Flex 容器（`display: flex`）、主轴（Main Axis）、交叉轴（Cross Axis）、Flex 项目。

容器属性：`flex-direction`、`justify-content`、`align-items`、`flex-wrap`、`align-content`、`gap`。

项目属性：`flex-grow`、`flex-shrink`、`flex-basis`、`flex`、`align-self`。

【操作步骤或工作流程】

步骤1：确定布局方向，设置 `flex-direction: row`（水平）或 `column`（垂直）。

步骤2：设置 `justify-content` 控制主轴对齐，如 `center` 居中、`space-between` 两端对齐。

步骤3：设置 `align-items` 控制交叉轴对齐，如 `center` 实现垂直居中。

步骤4：如果项目需要换行，设置 `flex-wrap: wrap`。

步骤5：使用 `gap` 设置项目间距，替代传统的 `margin` 方案。

步骤6：使用 `flex: 1` 让项目等分剩余空间，或使用 `flex: 0 0 200px` 固定大小。

步骤7：使用 `align-self` 让个别项目偏离容器的交叉轴对齐设置。

【注意事项】

1. 设置了 `display: flex` 的容器，其子元素会自动变成块级元素（blockified）。
2. `flex: 1` 等同于 `flex: 1 1 0%`，意味着项目基础大小为 0，完全按比例分配。
3. `align-content` 只在多行（`flex-wrap: wrap` 且发生换行）时生效，单行时无效。
4. `gap` 属性会同时设置主轴和交叉轴方向的间距。
5. 使用 `flex-wrap: wrap` 时，建议配合 `min-width` 或 `flex-basis` 防止项目过度压缩。

【常见问题】

Q1：为什么设置了 align-items: center 但没有垂直居中？

A：检查容器是否有明确的高度。如果容器高度由内容撑开，交叉轴居中效果不明显。

Q2：flex: 1 和 flex: auto 有什么区别？

A：flex: 1 是 flex: 1 1 0%，基础大小为 0；flex: auto 是 flex: 1 1 auto，基础大小由内容决定。

Q3：如何实现左侧固定、右侧自适应的布局？

A：左侧设置 flex: 0 0 250px（不放大不缩小固定 250px），右侧设置 flex: 1（占据剩余空间）。

【实际案例】

案例：电商网站导航栏与商品列表

```
.navbar {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
  padding: 0 20px;  
  height: 60px;  
  background: #1976d2;  
  color: white;  
}  
  
.navbar .logo { flex: 0 0 auto; }  
  
.navbar .nav-links { display: flex; gap: 24px; }  
  
.navbar .user-actions { flex: 0 0 auto; }  
  
.product-grid {  
  display: flex;  
  flex-wrap: wrap;  
  gap: 20px;
```

```
}

.product-grid > .product-card {

flex: 1 1 250px;

}

.layout {

display: flex;

gap: 20px;

min-height: 100vh;

}

.sidebar { flex: 0 0 240px; background: #f5f5f5; }

.main-content { flex: 1; }
```

3.2 Grid 网格布局

【知识点讲解】

CSS Grid 是二维布局系统，可以同时处理行和列，适合构建复杂的页面架构。与 Flexbox（一维）互补，Grid 更适合整体页面布局，Flexbox 更适合组件内部排列。

核心概念：Grid 容器（display: grid）、轨道（Track）、单元格（Cell）、区域（Area）。

容器属性：grid-template-columns、grid-template-rows、grid-template-areas、gap、justify-items、align-items、justify-content、align-content、grid-auto-flow。

项目属性：grid-column、grid-row、grid-area、justify-self、align-self。

常用单位：fr（fraction）、repeat()、minmax()、auto-fit、auto-fill。

【操作步骤或工作流程】

步骤1：分析页面布局，确定需要几行几列。

步骤2：使用 grid-template-columns 和 grid-template-rows 定义轨道大小。

步骤3：使用 repeat() 简化重复模式，如 repeat(12, 1fr) 创建 12 栏网格。

步骤4：使用 minmax() 实现响应式，如 minmax(250px, 1fr)。

步骤5：使用 grid-template-areas 通过命名区域直观定义布局。

步骤6：使用 grid-column 和 grid-row 让项目跨越多个单元格。

步骤7：使用 auto-fit 或 auto-fill 配合 minmax() 实现响应式自动填充。

【注意事项】

1. Grid 是二维布局，Flexbox 是一维布局，两者互补而非替代。
2. auto-fit 会拉伸项目填满容器，auto-fill 会保留空轨道，根据需求选择。
3. grid-template-areas 中的命名必须用引号包裹，每个区域名对应一个单元格。
4. 使用 grid-area 时，可以用 / 分隔四个值表示上、右、下、左的网格线位置。
5. 网格项目默认会拉伸填满单元格（stretch），可通过 justify-items/align-items 修改。

【常见问题】

Q1：Grid 和 Flexbox 如何选择？

A：整体页面架构用 Grid，组件内部排列用 Flexbox。需要同时控制行和列时用 Grid，只需控制一个方向用 Flexbox。

Q2：如何实现响应式卡片网格？

A：grid-template-columns: repeat(auto-fit, minmax(250px, 1fr))，卡片最小 250px，自动填充并等分剩余空间。

Q3：grid-area 的两种用法有什么区别？

A：grid-area: header 配合命名区域使用；grid-area: 1 / 2 / 3 / 4 直接指定四线位置（行起始/列起始/行结束/列结束）。

【实际案例】

案例：后台管理系统页面布局

```
.dashboard {  
  
  display: grid;  
  
  grid-template-columns: 240px 1fr 300px;  
  
  grid-template-rows: 64px 1fr auto;
```

```
grid-template-areas:

"header header header"

"sidebar main stats"

"footer footer footer";

min-height: 100vh;

gap: 0;

}

.dashboard header { grid-area: header; background: #1976d2; }

.dashboard aside { grid-area: sidebar; background: #f5f5f5; }

.dashboard main { grid-area: main; padding: 24px; }

.dashboard .stats { grid-area: stats; background: #fafafa; }

.dashboard footer { grid-area: footer; background: #e0e0e0; }

@media (max-width: 768px) {

.dashboard {

grid-template-columns: 1fr;

grid-template-rows: 64px auto 1fr auto auto;

grid-template-areas:

"header"

"sidebar"

"main"

"stats"

"footer";

}

}

.grid-12 {

display: grid;

grid-template-columns: repeat(12, 1fr);
```

```
gap: 16px;
```

```
}
```

```
.col-6 { grid-column: span 6; }
```

```
.col-4 { grid-column: span 4; }
```

```
.col-3 { grid-column: span 3; }
```

第4章 响应式设计媒体查询

4.1 响应式设计原理与断点策略

【知识点讲解】

响应式设计 (Responsive Design) 是指网页能够根据用户设备的屏幕尺寸、分辨率、方向等特性自动调整布局和样式。现代网站必须适配从手机到桌面的各种设备。

设计策略：移动优先 (Mobile First) 先为小屏幕设计，再通过 `min-width` 媒体查询逐步增强，现代推荐策略；桌面优先 (Desktop First) 先为大屏幕设计，再通过 `max-width` 媒体查询逐步简化。

常见断点：手机 $< 576\text{px}$ 、平板 $576\text{px}-991\text{px}$ 、小型桌面 $768\text{px}-991\text{px}$ 、桌面 $\geq 992\text{px}$ 、大屏 $\geq 1200\text{px}$ 、超大屏 $\geq 1400\text{px}$ 。

响应式单位：`vw/vh` (视口百分比)、`%` (相对于父元素)、`em/rem` (相对字体大小)、`clamp()` (设置值的范围)。

【操作步骤或工作流程】

- 步骤1：确定设计策略，推荐采用移动优先。
- 步骤2：编写基础样式，针对移动端 ($< 576\text{px}$) 进行设计。
- 步骤3：使用 `min-width` 媒体查询逐步增强大屏幕体验。
- 步骤4：设置常用断点： 576px 、 768px 、 992px 、 1200px 。
- 步骤5：使用相对单位 (`rem`、`%`、`vw`) 替代固定像素值。
- 步骤6：使用 `clamp()` 实现流畅的字体和间距缩放。
- 步骤7：测试不同设备和浏览器下的表现。

【注意事项】

- 1. 断点应基于内容而非设备，当布局开始"破裂"时添加断点。
- 2. 避免设置过多断点，通常 3-4 个即可覆盖大多数场景。
- 3. 使用 `rem` 单位时，建议在 `html` 上设置 `font-size: 62.5%` (即 10px)，便于计算。
- 4. 图片应设置 `max-width: 100%` 和 `height: auto`，防止超出容器。

5. 触摸设备上的交互元素最小尺寸应为 44x44px，便于点击。

【常见问题】

Q1：移动优先和桌面优先有什么区别？

A：移动优先先写移动端样式，用 `min-width` 增强；桌面优先先写桌面样式，用 `max-width` 简化。移动优先代码更少，性能更好。

Q2：如何测试响应式设计？

A：使用浏览器开发者工具的 Device Toolbar 模拟不同设备，同时应在真实设备上测试。Chrome DevTools 提供了多种预设设备尺寸。

Q3：`clamp()` 的浏览器兼容性如何？

A：现代浏览器均支持 `clamp()`，IE 不支持。如需兼容旧浏览器，可配合 `min()` 和 `max()` 或提供回退值。

【实际案例】

案例：响应式企业官网

```
.container {  
  width: 100%;  
  padding: 0 16px;  
}  
  
.hero-title {  
  font-size: clamp(28px, 5vw, 56px);  
  line-height: 1.2;  
}  
  
.feature-grid {  
  display: grid;  
  grid-template-columns: 1fr;  
  gap: 24px;  
}
```

```
@media (min-width: 576px) {  
  
  .feature-grid { grid-template-columns: repeat(2, 1fr); }  
  
}  
  
@media (min-width: 768px) {  
  
  .container { max-width: 720px; margin: 0 auto; }  
  
}  
  
@media (min-width: 992px) {  
  
  .container { max-width: 960px; }  
  
  .feature-grid { grid-template-columns: repeat(3, 1fr); }  
  
}  
  
@media (min-width: 1200px) {  
  
  .container { max-width: 1140px; }  
  
}
```

4.2 媒体查询与流体布局

【知识点讲解】

媒体查询 (Media Queries) 是响应式设计的核心技术，允许根据设备特性应用不同的 CSS 规则。

媒体查询语法：@media (条件) { ... }。常用条件：min-width、max-width、min-height、orientation (portrait/landscape)。逻辑操作：and、not、only、逗号 (或)。

高级媒体特性：prefers-color-scheme (检测系统主题 light/dark)、prefers-reduced-motion (检测用户是否偏好减少动画)、hover (检测设备是否支持悬停)、pointer (检测指针精度)。

资源预加载：<link rel="preconnect"> 提前建立连接、<link rel="preload"> 预加载关键资源、<link rel="prefetch"> 预加载后续页面资源。

【操作步骤或工作流程】

步骤1：使用 @media (min-width: 768px) 定义断点样式。

步骤2：使用 orientation 检测横竖屏，调整布局方向。

步骤3：使用 `prefers-color-scheme: dark` 提供深色模式样式。

步骤4：使用 `prefers-reduced-motion: reduce` 尊重用户的无障碍设置。

步骤5：使用 `<link rel="preconnect">` 提前连接第三方域名。

步骤6：使用 `<link rel="preload">` 预加载关键 CSS 和字体。

步骤7：使用 `srcset` 和 `sizes` 提供响应式图片。

【注意事项】

1. 媒体查询中的 `only` 关键字用于防止旧浏览器应用样式，现代浏览器可省略。
2. `prefers-reduced-motion` 应减少或消除动画，而非完全删除内容。
3. `preload` 应仅用于当前页面关键资源，过度使用会阻塞其他资源。
4. 打印样式 `@media print` 应隐藏导航、广告，调整字体和颜色为打印友好。
5. 逗号分隔多个媒体查询表示"或"关系，`and` 表示"与"关系。

【常见问题】

Q1：为什么媒体查询没有生效？

A：检查媒体查询条件是否正确，确保没有语法错误。注意 `min-width` 和 `max-width` 的区别，以及单位是否正确。

Q2：如何实现深色模式？

A：使用 `@media (prefers-color-scheme: dark)` 检测系统主题，或使用 CSS 变量配合 `class` 切换实现手动切换。

Q3：`preconnect` 和 `preload` 有什么区别？

A：`preconnect` 提前建立 TCP 连接和 TLS 握手；`preload` 提前下载指定资源。两者可配合使用。

【实际案例】

案例：支持深色模式的响应式博客

```
@media (prefers-color-scheme: dark) {  
  
  :root {  
  
    --bg-color: #1a1a1a;
```

```
--text-color: #e0e0e0;

--card-bg: #2d2d2d;

}

}

@media (prefers-color-scheme: light) {

:root {

--bg-color: #ffffff;

--text-color: #333333;

--card-bg: #f5f5f5;

}

}

body {

background: var(--bg-color);

color: var(--text-color);

}

@media (prefers-reduced-motion: reduce) {

*, *::before, *::after {

animation-duration: 0.01ms !important;

animation-iteration-count: 1 !important;

transition-duration: 0.01ms !important;

}

}

@media print {

.no-print { display: none; }

body { font-size: 12pt; color: black; }

a { text-decoration: none; color: black; }

}
```

```

```

第5章 JavaScript 核心基础

5.1 变量、数据类型与运算符

【知识点讲解】

JavaScript 是前端开发的核心编程语言，理解其基础语法是掌握前端开发的必经之路。

变量声明：var（函数作用域，可重复声明，存在变量提升）、let（块级作用域，不可重复声明，无变量提升）、const（块级作用域，声明时必须初始化，不可重新赋值但对象属性可修改）。

数据类型：原始类型（string、number、boolean、null、undefined、symbol、bigint）、引用类型（object，包括 Array、Function、Date、RegExp 等）。

类型检测：typeof（检测原始类型，对 null 返回 "object"）、instanceof（检测原型链）、Array.isArray()（专门检测数组）、Object.prototype.toString.call()（最准确的类型检测方法）。

运算符：算术运算符（+、-、*、/、%、**）、比较运算符（== 松散相等、=== 严格相等、!=、!==、>、<、>=、<=）、逻辑运算符（&&、||、!、?? 空值合并）、位运算符（&、|、^、~、<<、>>、>>>）。

【操作步骤或工作流程】

步骤1：优先使用 const 声明变量，只有需要重新赋值时才使用 let。

步骤2：避免使用 var，防止作用域污染和变量提升导致的意外行为。

步骤3：使用 === 和 !== 进行比较，避免类型转换带来的意外结果。

步骤4：使用 typeof 检测原始类型，使用 instanceof 检测自定义对象。

步骤5：使用 ?? 替代 || 进行默认值设置，避免 0、false、'' 被误判为空。

步骤6：使用模板字符串（`Hello \${name}`）替代字符串拼接。

步骤7：使用解构赋值简化变量提取。

【注意事项】

1. null 的 typeof 返回 "object"，这是 JavaScript 的历史 bug，不能用于检测 null。

2. NaN 与任何值都不相等，包括 NaN 本身，应使用 isNaN() 或 Number.isNaN() 检测。

3. $0.1 + 0.2 !== 0.3$ ，浮点数精度问题，涉及金额计算应使用整数分单位。
4. `const` 声明的对象不能重新赋值，但可以修改其属性，如需完全不可变应使用 `Object.freeze()`。
5. 模板字符串中的表达式会立即求值，不会动态更新。

【常见问题】

Q1：var、let、const 有什么区别？

A：var 是函数作用域且存在变量提升；let 和 const 是块级作用域且不存在变量提升。const 声明后不能重新赋值。

Q2：== 和 === 有什么区别？

A：== 会进行类型转换后比较，=== 要求类型和值都相同。推荐始终使用 ===。

Q3：如何判断一个变量是否为数组？

A：使用 `Array.isArray(arr)`，这是最可靠的方法。`typeof` 返回 "object"，`instanceof` 在跨 frame 时可能失效。

【实际案例】

案例：用户数据验证与处理

```
const API_BASE_URL = 'https://api.example.com';

let currentUser = null;

function validateAge(age) {
  if (typeof age !== 'number') return false;
  if (age < 0 || age > 150) return false;
  return Number.isInteger(age);
}

function getConfig(key) {
  const value = config[key] ?? defaultConfig[key] ?? '';
  return value;
}
```

```
function greetUser(user) {  
  return `欢迎回来，${user.name}！您有 ${user.unreadCount} 条未读消息。`;  
}  
  
const { id, name, email } = user;  
  
const [first, second, ...rest] = items;
```

5.2 流程控制、函数与作用域

【知识点讲解】

流程控制决定代码的执行路径，函数是代码复用的基本单元，作用域决定了变量的可见范围。

流程控制：条件（if/else、switch、三元运算符）、循环（for、while、do...while、for...of、for...in）、控制（break、continue、return、throw）。

函数：函数声明（function foo() {}，存在函数提升）、函数表达式（const foo = function() {}，不存在提升）、箭头函数（const foo = () => {}，没有自己的this、arguments、super）、默认参数、剩余参数、立即执行函数（IIFE）。

作用域：全局作用域、函数作用域、块级作用域（let/const 在 {} 内部）、作用域链（变量查找从内到外逐层向上）。

闭包：函数记住并访问其词法作用域，即使函数在当前作用域外执行。

【操作步骤或工作流程】

步骤1：使用 if/else 处理简单条件，使用 switch 处理多值匹配。

步骤2：使用 for...of 遍历可迭代对象（数组、字符串、Map、Set）。

步骤3：使用 for...in 遍历对象属性（注意要 hasOwnProperty 过滤）。

步骤4：优先使用箭头函数作为回调，避免 this 绑定问题。

步骤5：使用默认参数和剩余参数简化函数签名。

步骤6：使用 IIFE 创建私有作用域，避免污染全局命名空间。

步骤7：理解闭包机制，利用闭包实现数据私有化和函数工厂。

【注意事项】

1. switch 使用严格相等 (===) 进行比较，且不要忘记 break。
2. for...in 会遍历原型链上的可枚举属性，应配合 hasOwnProperty 使用。
3. 箭头函数不能作为构造函数使用（不能用 new），也没有 arguments 对象。
4. 函数参数按值传递，但对象参数传递的是引用地址。
5. 闭包会持有外部变量的引用，可能导致内存泄漏，注意及时释放。

【常见问题】

Q1：for...of 和 for...in 有什么区别？

A：for...of 遍历可迭代对象的值，适用于数组、字符串等；for...in 遍历对象的可枚举属性名，适用于普通对象。

Q2：箭头函数和普通函数有什么区别？

A：箭头函数没有自己的 this、arguments、super，不能作为构造函数，没有 prototype 属性。this 继承自外层作用域。

Q3：什么是闭包？有什么应用场景？

A：闭包是函数及其词法环境的组合。应用场景包括：数据私有化、函数工厂、回调函数保持状态、防抖节流等。

【实际案例】

案例：购物车模块的函数设计

```
function createCounter(initial = 0) {  
  let count = initial;  
  return {  
    increment() { return ++count; },  
    decrement() { return --count; },  
    getValue() { return count; },  
    reset() { count = initial; return count; }  
  };  
}
```

```
}

const cartCounter = createCounter(0);

cartCounter.increment();

cartCounter.increment();

function debounce(fn, delay) {

  let timer = null;

  return function(...args) {

    clearTimeout(timer);

    timer = setTimeout(() => fn.apply(this, args), delay);

  };

}

const searchInput = document.getElementById('search');

searchInput.addEventListener('input', debounce((e) => {

  console.log('搜索:', e.target.value);

}, 300));

function createUser(name, role = 'user', ...permissions) {

  return { name, role, permissions };

}
```

第6章 ES6+ 新特性与异步编程

6.1 ES6+ 语法新特性

【知识点讲解】

ES6 (ECMAScript 2015) 及后续版本为 JavaScript 带来了大量现代化语法特性，极大地提升了代码的可读性和开发效率。

核心新特性：解构赋值、展开运算符 (...)、类 (Class)、模块 (Module import/export)、Promise、异步函数 (async/await)、可选链 (?.)、空值合并 (??)、逻辑赋值 (||=、&&=、??=)。

数组和对象新方法：Array (map、filter、reduce、find、findIndex、includes、flat、flatMap)、Object (assign、entries、values、keys、fromEntries)、String (includes、startsWith、endsWith、repeat、padStart、padEnd)。

【操作步骤或工作流程】

- 步骤1：使用 let/const 替代 var，使用箭头函数简化回调。
- 步骤2：使用解构赋值从对象/数组中提取数据。
- 步骤3：使用展开运算符复制对象/数组或合并多个对象/数组。
- 步骤4：使用模板字符串替代字符串拼接。
- 步骤5：使用类语法定义构造函数和方法。
- 步骤6：使用 import/export 组织代码模块。
- 步骤7：使用数组新方法 (map、filter、reduce) 替代传统循环。

【注意事项】

1. 展开运算符是浅拷贝，嵌套对象/数组仍共享引用。
2. 类中的方法默认不在原型上 (使用 class fields 语法时)，注意 this 绑定。
3. 模块化代码在浏览器中需要 type="module" 或构建工具处理。
4. 可选链不能用于赋值左侧，如 obj?.prop = value 会报错。
5. forEach 中不能使用 await，应使用 for...of 配合 await。

【常见问题】

Q1：展开运算符和 Object.assign 有什么区别？

A：两者都是浅拷贝。展开运算符更简洁，且可以覆盖属性；Object.assign 会修改第一个参数对象。

Q2：类中的箭头函数方法和普通方法有什么区别？

A：箭头函数方法绑定在实例上，不共享原型；普通方法在原型上，所有实例共享。箭头函数方法内存占用更大。

Q3：如何深拷贝一个对象？

A：使用 `JSON.parse(JSON.stringify(obj))` 进行简单深拷贝（不能处理函数、Date、循环引用）。复杂场景使用 `lodash.cloneDeep` 或 `structuredClone`。

【实际案例】

案例：用户信息处理模块

```
const user = { id: 1, name: '张三', email: 'zhang@example.com', role: 'admin' };
```

```
const { name, ...rest } = user;
```

```
const updatedUser = { ...rest, name: '张三（已更新）', lastLogin: new Date() };
```

```
const orders = [
```

```
{ id: 1, amount: 150, status: 'completed' },
```

```
{ id: 2, amount: 200, status: 'pending' },
```

```
{ id: 3, amount: 100, status: 'completed' }
```

```
];
```

```
const completedTotal = orders
```

```
.filter(order => order.status === 'completed')
```

```
.map(order => order.amount)
```

```
.reduce((sum, amount) => sum + amount, 0);
```

```
class User {
```

```

constructor(name, email) {

  this.name = name;

  this.email = email;

}

greet() {

  return `Hello, ${this.name}`;

}

static createGuest() {

  return new User('Guest', '');

}

}

const city = user?.address?.city ?? '未知城市';

```

6.2 异步编程与 Promise

【知识点讲解】

JavaScript 是单线程语言，异步编程是处理网络请求、文件操作、定时任务等 I/O 操作的核心机制。

异步编程演进：回调函数（最早的异步处理方式，容易导致回调地狱）、Promise（代表异步操作最终完成或失败的对象）、async/await（基于 Promise 的语法糖，使异步代码看起来像同步代码）。

Promise 状态：pending（初始状态）、fulfilled（操作成功完成）、rejected（操作失败）。

Promise 方法：then() 处理成功状态、catch() 处理失败状态、finally() 无论成功失败都执行、Promise.all() 等待所有 Promise 完成、Promise.race() 返回最快完成的 Promise、Promise.allSettled() 等待所有 Promise settled、Promise.any() 返回第一个成功的 Promise。

async/await：async 函数总是返回 Promise、await 只能在 async 函数内部使用、try/catch 处理异步错误。

【操作步骤或工作流程】

步骤1：将回调式 API 包装为 Promise，如 `new Promise((resolve, reject) => { ... })`。

步骤2：使用 `.then().catch()` 链式处理异步操作。

步骤3：使用 `async/await` 替代复杂的 Promise 链，提升可读性。

步骤4：使用 `Promise.all()` 并行执行多个独立的异步操作。

步骤5：使用 `try/catch` 包裹 `await` 代码，统一处理错误。

步骤6：在 `finally` 中执行清理操作（如关闭加载状态）。

步骤7：避免在 `forEach` 中使用 `await`，改用 `for...of` 或 `Promise.all()`。

【注意事项】

1. `await` 会暂停 `async` 函数的执行，但不会阻塞主线程。
2. Promise 链中的错误会向下传递，直到被 `catch` 捕获。
3. `async` 函数中未捕获的错误会返回 `rejected Promise`。
4. `Promise.all()` 中任一 Promise 失败，整体立即失败。
5. 不要在非 `async` 函数中使用 `await`，除非在支持顶层 `await` 的环境中。

【常见问题】

Q1：Promise 和 `async/await` 有什么区别？

A：`async/await` 是基于 Promise 的语法糖，功能等价但可读性更好。`async/await` 使异步代码看起来像同步代码，更易于理解和维护。

Q2：如何并行执行多个异步请求？

A：使用 `Promise.all([req1, req2, req3])`，所有请求同时发起，等待全部完成。如果请求之间独立，这比顺序 `await` 快得多。

Q3：`async/await` 中的错误如何处理？

A：使用 `try/catch` 包裹 `await` 语句。也可以在 `async` 函数调用处使用 `.catch()`。

【实际案例】

案例：电商网站数据获取与处理

```
async function loadProductPage(productId) {

  try {

    const [product, recommendations] = await Promise.all([

      fetch(`/api/products/${productId}`).then(r => r.json()),

      fetch(`/api/products/${productId}/recommendations`).then(r => r.json())

    ]);

    renderProduct(product);

    renderRecommendations(recommendations);

  } catch (error) {

    console.error('加载商品失败:', error);

    showErrorMessage('商品加载失败，请稍后重试');

  } finally {

    hideLoadingSpinner();

  }

}

async function loadUserDashboard() {

  try {

    const user = await fetch('/api/user').then(r => r.json());

    const orders = await fetch(`/api/users/${user.id}/orders`).then(r => r.json());

    const stats = await fetch(`/api/users/${user.id}/stats`).then(r => r.json());

    return { user, orders, stats };

  } catch (error) {

    console.error('加载仪表盘失败:', error);

    throw error;

  }

}

async function fetchWithRetry(url, options = {}, maxRetries = 3) {
```

```
for (let i = 0; i < maxRetries; i++) {  
  try {  
    const response = await fetch(url, options);  
    if (!response.ok) throw new Error(`HTTP ${response.status}`);  
    return response;  
  } catch (error) {  
    if (i === maxRetries - 1) throw error;  
    await new Promise(resolve => setTimeout(resolve, 1000 * (i + 1)));  
  }  
}
```

第7章 React 框架基础

7.1 React 组件与 JSX

【知识点讲解】

React 是由 Meta (Facebook) 开发的用于构建用户界面的 JavaScript 库。核心思想是组件化和声明式编程，让开发者专注于"UI 应该是什么样子"而非"如何操作 DOM"。

JSX (JavaScript XML) : React 的语法扩展，允许在 JS 中编写类似 HTML 的结构。编译后转换为 `React.createElement()` 调用。必须有一个根元素 (或使用 `Fragment` `<></>`)。使用 `{}` 嵌入 JavaScript 表达式。`className` 代替 `class`，`htmlFor` 代替 `for`。标签必须闭合。

组件类型：函数组件 (接收 `props` 返回 JSX，推荐 React 16.8+)、类组件 (继承 `React.Component`，使用 `render()` 方法，已较少使用)。组件命名必须以大写字母开头 (区分于 HTML 标签)。

条件渲染：三元运算符 `{isLoggedIn ? <Dashboard /> : <Login />}`、逻辑与 `{isAdmin && <AdminPanel />}`、提前返回 `if (!user) return <Login />`。

列表渲染：使用 `map()` 遍历数组、必须提供 `key` 属性帮助 React 识别元素变化、`key` 应在兄弟元素中唯一推荐使用稳定 ID 而非索引。

【操作步骤或工作流程】

步骤1：分析 UI，将其拆分为独立的、可复用的组件层次结构。

步骤2：使用函数组件定义每个组件，接收 `props` 返回 JSX。

步骤3：使用 JSX 编写组件的 UI 结构，注意使用 `className` 替代 `class`。

步骤4：使用 `{}` 嵌入 JavaScript 表达式，如变量、函数调用、三元运算。

步骤5：使用条件渲染控制元素的显示与隐藏。

步骤6：使用 `map()` 渲染列表，为每个列表项提供唯一的 `key`。

步骤7：使用 `Fragment` (`<></>`) 包裹多个根元素，避免不必要的 DOM 层级。

【注意事项】

1. JSX 中的表达式不能包含语句（如 if、for），只能使用表达式。
2. false、null、undefined 不会渲染任何内容，但 0 和空字符串会渲染。
3. 事件处理使用驼峰命名（onClick 而非 onclick），传入函数而非字符串。
4. style 属性接受对象而非字符串，且属性名使用驼峰命名。
5. 组件必须纯函数：相同的输入（props）总是产生相同的输出（JSX）。

【常见问题】

Q1：为什么列表渲染需要 key？

A：key 帮助 React 识别哪些元素改变了、添加了或删除了。没有 key 时 React 默认使用索引，但索引在列表顺序变化时会导致性能问题和状态错误。

Q2：JSX 中的 className 和 htmlFor 是什么？

A：class 和 for 是 JavaScript 的保留字，JSX 中使用 className 和 htmlFor 替代，编译后转换为 class 和 for 属性。

Q3：为什么组件名必须大写？

A：React 使用首字母大小写区分组件和 HTML 标签。小写标签被视为 HTML 元素，大写标签被视为自定义组件。

【实际案例】

案例：电商网站商品卡片组件

```
function ProductCard({ product }) {  
  const { id, name, price, image, rating, isNew } = product;  
  return (  
    <div className="product-card">  
      {isNew && <span className="badge-new">新品</span>}  
      <img src={image} alt={name} className="product-image" />  
      <div className="product-info">  
        <h3 className="product-name">{name}</h3>  

```

```

<div className="product-rating">
  {'★'.repeat(rating)}{'☆'.repeat(5 - rating)}
  <span>({rating}/5)</span>
</div>

<p className="product-price">¥{price.toFixed(2)}</p>

<button className="btn-primary" onClick={() => addToCart(id)}>
  加入购物车
</button>
</div>
</div>
);
}

```

```

function ProductList({ products, category }) {
  const filteredProducts = category
    ? products.filter(p => p.category === category)
    : products;

  if (filteredProducts.length === 0) {
    return <div className="empty-state">暂无商品</div>;
  }

  return (
    <div className="product-grid">
      {filteredProducts.map(product => (
        <ProductCard key={product.id} product={product} />
      ))}
    </div>
  );
}

```

7.2 Props、State 与 Hooks

【知识点讲解】

Props 和 State 是 React 组件的两大核心概念，Hooks 则让函数组件拥有了状态和其他 React 特性。

Props (Properties)：父组件向子组件传递数据的方式、只读 (Read-only) 子组件不能修改 props、可以是任意 JavaScript 值 (字符串、数字、对象、数组、函数)。

State (状态)：组件内部管理的数据、使用 `useState` Hook 声明和更新、State 更新是异步的可能会被批量处理、更新函数式 state 时应使用函数形式 `setCount(c => c + 1)`。

常用 Hooks：`useState` (管理状态)、`useEffect` (处理副作用，数据获取、订阅、DOM 操作)、`useRef` (获取 DOM 元素引用或保存可变值)、`useMemo` (缓存计算结果)、`useCallback` (缓存函数引用)。

`useEffect` 详解：`[]` 只在挂载和卸载时执行、`[a, b]` 在挂载和依赖变化时执行、省略每次渲染后都执行、返回清理函数在卸载或依赖变化前执行。

Hooks 使用规则：只在最顶层调用 Hooks 不在循环条件或嵌套函数中调用、只在 React 函数组件或自定义 Hooks 中调用。

【操作步骤或工作流程】

步骤1：使用 `useState` 声明组件状态，提供初始值。

步骤2：使用 state 值渲染 UI，通过事件处理函数更新 state。

步骤3：使用 `useEffect` 处理副作用，如数据获取、订阅、手动 DOM 操作。

步骤4：在 `useEffect` 中指定依赖数组，控制执行时机。

步骤5：在 `useEffect` 中返回清理函数，防止内存泄漏。

步骤6：使用 `useRef` 获取 DOM 元素引用或保存不触发渲染的值。

步骤7：使用 `useMemo` 和 `useCallback` 优化性能，避免不必要的计算和渲染。

【注意事项】

1. 不要直接修改 state，应使用 `setState` 函数。对象/数组 state 应创建新引用。

2. `useEffect` 的依赖数组应包含所有在 `effect` 中使用的响应式值。
3. 空依赖数组 `[]` 的 `useEffect` 只在挂载和卸载时执行，类似于 `componentDidMount`。
4. `useRef` 的 `.current` 属性修改不会触发重新渲染。
5. 自定义 Hooks 应以 `use` 开头命名，如 `useLocalStorage`。

【常见问题】

Q1：为什么 `state` 更新后没有立即生效？

A：`setState` 是异步的，`React` 会批量处理 `state` 更新以优化性能。如需使用最新值，可在 `useEffect` 中监听，或使用函数式更新。

Q2：`useEffect` 中遗漏依赖会有什么后果？

A：`effect` 会使用过期的闭包值，导致 `bug`。`React` 的 `eslint-plugin-react-hooks` 插件会检测缺失的依赖，应遵循其提示。

Q3：什么时候使用 `useMemo` 和 `useCallback`？

A：当计算成本较高或需要将对象/函数作为 `props` 传递给子组件（配合 `React.memo`）时使用。过度使用反而增加开销。

【实际案例】

案例：用户登录与个人信息管理

```
import { useState, useEffect, useRef } from 'react';
```

```
function UserProfile({ userId }) {
```

```
  const [user, setUser] = useState(null);
```

```
  const [loading, setLoading] = useState(true);
```

```
  const [error, setError] = useState(null);
```

```
  const inputRef = useRef(null);
```

```
  useEffect(() => {
```

```
    let cancelled = false;
```

```
    async function fetchUser() {
```

```
try {  
  
  setLoading(true);  
  
  const response = await fetch(`/api/users/${userId}`);  
  
  if (!response.ok) throw new Error('获取失败');  
  
  const data = await response.json();  
  
  if (!cancelled) {  
  
    setUser(data);  
  
    setError(null);  
  
  }  
  
  } catch (err) {  
  
    if (!cancelled) setError(err.message);  
  
  } finally {  
  
    if (!cancelled) setLoading(false);  
  
  }  
  
  }  
  
  fetchUser();  
  
  return () => { cancelled = true; };  
  
  }, [userId]);  
  
const handleUpdateName = async (newName) => {  
  
  try {  
  
    const response = await fetch(`/api/users/${userId}`, {  
  
      method: 'PATCH',  
  
      headers: { 'Content-Type': 'application/json' },  
  
      body: JSON.stringify({ name: newName })  
  
    });  
  
    const updated = await response.json();  
  
    setUser(prev => ({ ...prev, name: updated.name }));  
  
  }
```

```
} catch (err) {  
  
  setError('更新失败');  
  
}  
  
};  
  
if (loading) return <div>加载中...</div>;  
if (error) return <div>错误: {error}</div>;  
if (!user) return <div>用户不存在</div>;  
  
return (  
  
  <div className="user-profile">  
  
    <h2>{user.name}</h2>  
  
    <p>{user.email}</p>  
  
    <input ref={inputRef} defaultValue={user.name} />  
  
    <button onClick={() => handleUpdateName(inputRef.current.value)}>  
      更新名称  
    </button>  
  
  </div>  
  
);  
}
```

第8章 前端工程化与性能优化

8.1 构建工具与包管理

【知识点讲解】

前端工程化是现代前端开发的基石，涵盖包管理、构建工具、代码规范、版本控制等方面，目的是提升开发效率、代码质量和协作能力。

npm (Node Package Manager) : package.json (项目配置文件)、node_modules (存放下载的依赖包)、package-lock.json (锁定依赖的确切版本和依赖树结构)、语义化版本 SemVer (主版本.次版本.修订号)。

常用命令：npm init -y (快速创建 package.json)、npm install <pkg> (安装包到 dependencies)、npm install -D <pkg> (安装到 devDependencies)、npm install -g <pkg> (全局安装)、npx <cmd> (执行 node_modules 中的命令)。

构建工具：Webpack (功能强大、生态丰富的老牌打包工具，支持 Loader 和 Plugin)、Vite (新一代构建工具，开发阶段基于原生 ES Module，启动极快)、代码分割 (按需加载，减少初始加载时间)、Tree Shaking (消除未使用的代码)。

代码规范：ESLint (JavaScript 代码检查工具)、Prettier (代码格式化工具)、Husky + lint-staged (Git 提交前自动运行代码检查)。

Git 版本控制：工作区、暂存区、本地仓库、远程仓库。分支策略：main (生产)、develop (开发)、feature/* (功能分支)。

【操作步骤或工作流程】

步骤1：使用 npm init -y 初始化项目，创建 package.json。

步骤2：安装项目依赖 (react、lodash 等) 和开发依赖 (vite、eslint 等)。

步骤3：配置构建工具 (Vite 或 Webpack)，设置入口、输出、Loader、Plugin。

步骤4：配置 ESLint 和 Prettier，统一代码风格。

步骤5：配置 Husky 和 lint-staged，在 pre-commit 钩子中运行代码检查。

步骤6：使用 Git 初始化仓库，建立 main 和 develop 分支。

步骤7：功能开发使用 feature 分支，完成后通过 Pull Request 合并。

【注意事项】

1. dependencies 和 devDependencies 的区别：生产环境部署时只安装 dependencies。
2. ^2.3.4 允许更新到同一主版本的最新次版本 ($\geq 2.3.4 < 3.0.0$)，~2.3.4 只允许修订号更新。
3. package-lock.json 应提交到版本控制，确保团队成员安装一致的依赖。
4. ESLint 和 Prettier 可能冲突，使用 eslint-config-prettier 关闭冲突规则。
5. Git 提交信息应遵循规范（如 Conventional Commits），便于生成 CHANGELOG。

【常见问题】

Q1：npm install 和 npm ci 有什么区别？

A：npm install 根据 package.json 安装并可能更新 package-lock.json；npm ci 严格根据 package-lock.json 安装，删除 node_modules 后重新安装，适合 CI/CD 环境。

Q2：Vite 和 Webpack 如何选择？

A：新项目推荐 Vite，启动快、配置简单；大型成熟项目或需要复杂定制时可选 Webpack。

Q3：如何解决 ESLint 和 Prettier 的冲突？

A：安装 eslint-config-prettier 和 eslint-plugin-prettier，在 .eslintrc 中 extends 添加 'prettier'，关闭 ESLint 的格式规则，让 Prettier 负责格式化。

【实际案例】

案例：React 项目工程化配置

```
// package.json

{
  "name": "ecommerce-frontend",
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
```

```
"build": "vite build",

"preview": "vite preview",

"test": "vitest",

"lint": "eslint src --ext .js,.jsx",

"lint:fix": "eslint src --ext .js,.jsx --fix",

"format": "prettier --write "src/**/*.{js,jsx,css,md}""

},

"dependencies": {

"react": "^18.2.0",

"react-dom": "^18.2.0",

"react-router-dom": "^6.20.0"

},

"devDependencies": {

"@vitejs/plugin-react": "^4.2.0",

"eslint": "^8.55.0",

"eslint-config-prettier": "^9.1.0",

"eslint-plugin-react": "^7.33.0",

"prettier": "^3.1.0",

"vite": "^5.0.0",

"vitest": "^1.0.0"

}

}
```

```
// vite.config.js
```

```
import { defineConfig } from 'vite';
```

```
import react from '@vitejs/plugin-react';
```

```
import path from 'path';
```

```
export default defineConfig({
```

```
plugins: [react()],

resolve: {

  alias: { '@': path.resolve(__dirname, './src') }

},

build: { outDir: 'dist', sourcemap: true }

});

// .eslintrc.cjs

module.exports = {

  env: { browser: true, es2021: true },

  extends: [

    'eslint:recommended',

    'plugin:react/recommended',

    'plugin:react-hooks/recommended',

    'prettier'

  ],

  parserOptions: { ecmaVersion: 'latest', sourceType: 'module' },

  rules: {

    'react/react-in-jsx-scope': 'off',

    'no-console': 'warn'

  }

};
```

8.2 前端性能优化与安全

【知识点讲解】

前端性能直接影响用户体验和转化率，安全则是保障用户数据和系统稳定的底线。

Core Web Vitals : LCP (Largest Contentful Paint，最大内容绘制，衡量加载性能，目标 < 2.5s)、INP (Interaction to Next Paint，交互响应延迟，目标 < 200ms)、CLS (Cumulative

Layout Shift，累积布局偏移，衡量视觉稳定性，目标 < 0.1）。

性能优化策略：加载优化（代码分割、图片优化 WebP/懒加载、资源预加载、CDN、Gzip/Brotli 压缩）、渲染优化（避免强制同步布局、使用 transform 和 opacity 实现动画 GPU 加速、减少重排和重绘）、缓存策略（HTTP 缓存 Cache-Control/ETag、Service Worker 离线缓存）。

安全基础：XSS（跨站脚本攻击，注入恶意脚本，防御：输出编码、CSP、HttpOnly Cookie）、CSRF（跨站请求伪造，诱导已登录用户执行非预期操作，防御：CSRF Token、SameSite Cookie）。

【操作步骤或工作流程】

步骤1：使用 Lighthouse 或 PageSpeed Insights 检测性能指标。

步骤2：实施代码分割，使用 React.lazy 和 Suspense 实现路由级懒加载。

步骤3：优化图片：使用 WebP 格式、响应式图片 srcset、懒加载 loading="lazy"。

步骤4：配置资源预加载：preconnect 第三方域名、preload 关键 CSS 和字体。

步骤5：配置 HTTP 缓存策略：静态资源使用 Cache-Control: max-age=31536000 并配合文件名哈希。

步骤6：防御 XSS：对用户输入进行转义，使用textContent 替代 innerHTML，设置 Content-Security-Policy。

步骤7：防御 CSRF：在表单和 AJAX 请求中添加 CSRF Token，设置 Cookie 的 SameSite 属性。

【注意事项】

1. LCP 优化应优先关注首屏最大图片或文本块的加载速度。
2. CLS 优化应确保图片和视频有明确的 width 和 height 属性，避免布局跳动。
3. 代码分割应适度，过度分割会增加 HTTP 请求数量，反而降低性能。
4. CSP 策略应从宽松开始逐步收紧，避免阻断正常功能。
5. SameSite=Strict 可能破坏跨站链接的登录状态，需根据业务场景选择 Strict/Lax/None。

【常见问题】

Q1：如何优化首屏加载时间？

A：关键路径优化：内联关键 CSS、延迟加载非关键 JS、使用字体显示策略 `font-display: swap`、预加载 LCP 图片。

Q2：什么是重排（Reflow）和重绘（Repaint）？如何减少？

A：重排是几何属性变化导致重新计算布局；重绘是外观变化不影响布局。减少方法：批量修改样式（使用 `class` 替代逐个修改 `style`）、使用 `transform` 替代 `top/left`、避免在循环中读取 `offsetHeight` 等触发强制同步布局的属性。

Q3：XSS 和 CSRF 有什么区别？

A：XSS 是攻击者将恶意脚本注入到网页中执行，利用的是网站对用户输入的信任；CSRF 是诱导已登录用户访问恶意链接，利用的是浏览器自动携带 `Cookie` 的特性。

【实际案例】

案例：电商网站性能与安全优化

// 代码分割与懒加载

```
import { lazy, Suspense } from 'react';

const ProductDetail = lazy(() => import('./pages/ProductDetail'));

const Cart = lazy(() => import('./pages/Cart'));

function App() {

  return (

    <Suspense fallback={<div>加载中...</div>}>

      <Routes>

        <Route path="/product/:id" element={<ProductDetail />} />

        <Route path="/cart" element={<Cart />} />

      </Routes>

    </Suspense>

  );

}
```

// 图片优化

```

```

```
// XSS 防御
```

```
function SafeComment({ text }) {
// 使用 textContent 而非 innerHTML
return <div className="comment">{text}</div>;
}
```

```
// CSRF 防御
```

```
async function apiRequest(url, data) {
const csrfToken = document.querySelector('meta[name="csrf-token"]')?.content;
const response = await fetch(url, {
method: 'POST',
headers: {
'Content-Type': 'application/json',
'X-CSRF-Token': csrfToken
},
credentials: 'same-origin',
body: JSON.stringify(data)
});
return response;
}
```

```
// Service Worker 缓存
```

```
const CACHE_NAME = 'app-v1';
self.addEventListener('install', event => {
event.waitUntil(
```

```
    caches.open(CACHE_NAME).then(cache =>
    cache.addAll(['/', '/styles.css', '/app.js'])
    )
  );
});

self.addEventListener('fetch', event => {
  event.respondWith(
    caches.match(event.request).then(response =>
    response || fetch(event.request)
    )
  );
});
```